

19×19 六子棋程序对战系统

比赛版游戏设计文档

版本：V1.0 实现语言：Python 适用场景：两个独立棋手程序自动对局

1. 项目目标

本项目实现一个用于程序对抗赛的六子棋系统。棋盘采用 19×19 规格，黑方首回合落一子，之后黑白双方轮流每回合落两子。横、竖或两条斜线连续六子及以上即获胜；棋盘填满且无人获胜时判和棋。

比赛由裁判程序统一负责颜色随机分配、棋盘状态、原子回合验证、棋钟、点位同步及胜负判定。A、B 是两个相互独立的棋手程序，界面可以不同，只通过统一通信协议交换比赛数据。

2. 规则与比赛约定

项目	约定
棋盘	19×19, 列 A-S, 行 1-19
首回合	随机获得黑方的一方落一子
普通回合	双方轮流各落两子
原子提交	两颗棋子必须一次性提交；任意一颗非法则整回合无效
胜负	连续六子或更长即获胜，长连有效
禁手	无禁手，三三、四四等均允许
修改限制	成功落子后不可修改、撤回、覆盖或移动
和棋	棋盘填满且没有六连

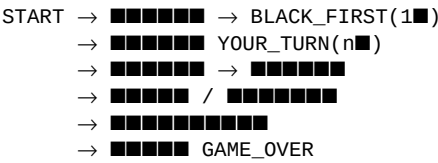
3. 系统架构

系统采用裁判服务器加两个棋手客户端的结构。裁判维护唯一的完整棋盘，双方程序不读取对方界面，只接收裁判广播的落子点位。



建议使用 TCP Socket 传输 JSON 行消息。这样可以支持同机对战，也可以扩展到局域网双机比赛。

4. 比赛状态机



裁判收到完整坐标列表后，先复制棋盘进行验证；只有两颗棋子全部合法，才一次性替换正式棋盘。验证期间正式棋盘不改变，也不会向对方发送中间状态。

5. 通信协议

消息	方向	示例
GAME_START	裁判→棋手	{ "color": "BLACK", "board_size": 19 }
YOUR_TURN	裁判→当前棋手	{ "stones": 2, "time_left": 597.4 }
TURN	棋手→裁判	{ "moves": ["J10", "K11"] }
TURN_ACCEPTED	裁判→双方	{ "color": "WHITE", "moves": ["J10", "K11"] }
TURN_REJECTED	裁判→当前棋手	{ "reason": "位置已被占用" }

GAME_OVER	裁判→双方	{"winner":"BLACK"}
-----------	-------	--------------------

每次成功提交后，裁判向双方广播本回合全部点位。双方程序据此更新本地棋盘；本地状态与裁判冲突时应记录错误并停止决策，不得自行猜测。

6. Python 核心模块

推荐目录如下:

```
six_stones_match/  
■■■ referee.py      # ■■■■■  
■■■ board.py        # 19x19■■■■■■■■■■  
■■■ rules.py         # ■■■■■■■■  
■■■ turn.py          # ■■■■■■■■■■  
■■■ clock.py         # ■■■■  
■■■ protocol.py     # JSON■■■  
■■■ player_a.py     # ■■■■  
■■■ player_b_test.py # ■■■■  
■■■ replay.py       # ■■■■
```

六连检查只需扫描横、竖、主对角线和副对角线四个方向，判断连续数量是否大于等于 6。棋盘只提供 place() 操作，禁止覆盖、删除和移动。

7. 棋钟与超时

双方各有独立棋钟。轮到某方时启动该方计时器；裁判收到完整回合并完成处理后停止计时。思考、生成两颗点位、通信和非法回合重提时间均计入本方用时。剩余时间小于等于零时立即判负。

```
clock.start()
receive complete TURN
clock.stop()
if clock.remaining() <= 0: winner = opponent
```

8. 我方棋手自动决策

我方程序收到 YOUR TURN

后，根据本地完整同步棋盘自动计算一颗或两颗点位，并在一次消息中提交。AI 可以分阶段实现：先做随机落子，再加入立即成六、阻止对手成六、五连、活四、活三和中心/邻近度评分，最后使用候选点筛选加 Minimax/Negamax 搜索。

建议候选点只取已有棋子附近 2 格以内的位置, 显著降低两子组合的搜索分支。

9. 日志、复盘与测试

每个成功回合追加写入不可变棋谱，记录回合号、颜色、两颗点位、时间和剩余用时。非法回合只写入调试日志，不写入正式棋谱。

测试重点包括：黑方首回合数量、同回合重复点、整回合原子失败、长连胜利、棋盘填满和棋、超时判负、颜色随机分配、双方点位广播一致性。

10. 实施计划

第一阶段完成棋盘、坐标转换和六连判断；第二阶段完成裁判服务器、JSON 协议和原子回合；第三阶段加入棋钟、超时和日志；第四阶段实现随机/启发式棋手并进行自动对局；第五阶段加入图形界面与复盘工具。

最终比赛时只启动裁判程序、我方棋手程序和对方棋手程序。颜色由裁判随机决定，双方按照协议自动完成整局比赛。